

Network Security

Devashish Gosain

Learning Objectives

At the end of this course, you will be able to:

- Describe the relevant layers of network communication
- Authentication, e.g., Kerberos
- Cryptographic subsystems (e.g. OpenSSL) – how to (or not to) use them.
- Authentication subsystems in modern systems – how to (or not to) use them.
- Network security postures – access controls (aka firewalls) , L3/L5 security.
- SSL/TLS
- VPNs/Proxies

This Course is NOT about

- Using LLM to learn hacking
- AI, e.g., ChatGPT
- Flaunting that I know Security

You Should Take This Class IFF...

- You want to learn about building secure software systems and networks.
- **You are not afraid to program in C/C++ and have used written systems programs in the past using OS system calls (preferably of some UNIXen), sockets API.**
- (Desirable pre-requisite): Some computer security class, e.g. Cryptography.

General Information

- Tutors/TA
 - Roshan Jha
 - Sachin Parsuraman Mudaliyar
 - Arghyadip
- Contact them via Email

Course Eval

- Two Quizzes (10%)
- Three Assignments (30%)
- Mid-Term (20%)
- End-term (40%)

Background reading

- Network Security. PRIVATE Communication in a PUBLIC World. Charlie Kaufman, Radia Perlman and Mike Speciner. PHI Learning Pvt. Ltd. 2nd Edition, ASIN : B001ADIWNI, ISBN 978-81-203-2213-4
- Computer Security: Principles and Practices (2nd Edition), W. Stallings and L. Browne, 2011, ISBN-13: 978-0132775069 (Selected Topics)
- Note there is no one book that covers all. We will refer to research papers published at reputed venues like IMC, NDSS, Usenix Security and PoPETS.

Outline

- Basic cryptographic primitives
- Studying few existing protocols
- Advanced cryptographic protocols with privacy applications
- Example applications

Cryptology

- Cryptology is a science that studies:
 - Cryptography (“**secret writing**”): Making secret messages
 - Turning **plaintext** (an ordinary readable message) into **ciphertext** (secret messages that are “hard” to read)
 - Cryptanalysis: Breaking secret messages
 - Recovering the plaintext from the ciphertext
- The point of cryptography is to enable sending secure messages over an insecure medium (such as the Internet)
- Cryptanalysis studies cryptographic systems to look for weaknesses or leaks of information

Scope

- The goal of the cryptography unit in this course is to show you what cryptographic tools exist, and information about using these tools in a secure manner
- Won't be showing you details of how the tools work
 - Refer specialized crypto courses or books for that

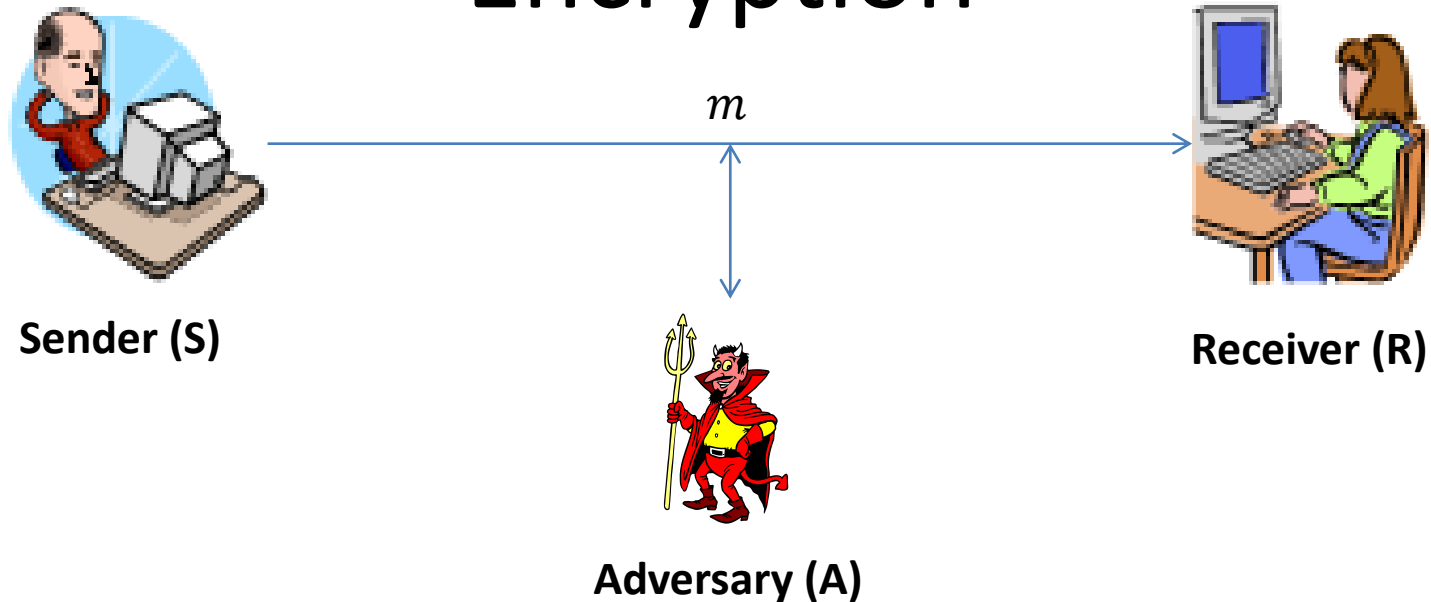
Building Blocks

Cryptography contains three major types of components

- Confidentiality components
 - Preventing Eve from **reading** Alice's messages
- Integrity components
 - Preventing Mallory from **modifying** Alice's messages without being detected
- Availability

Often remembered as CIA.

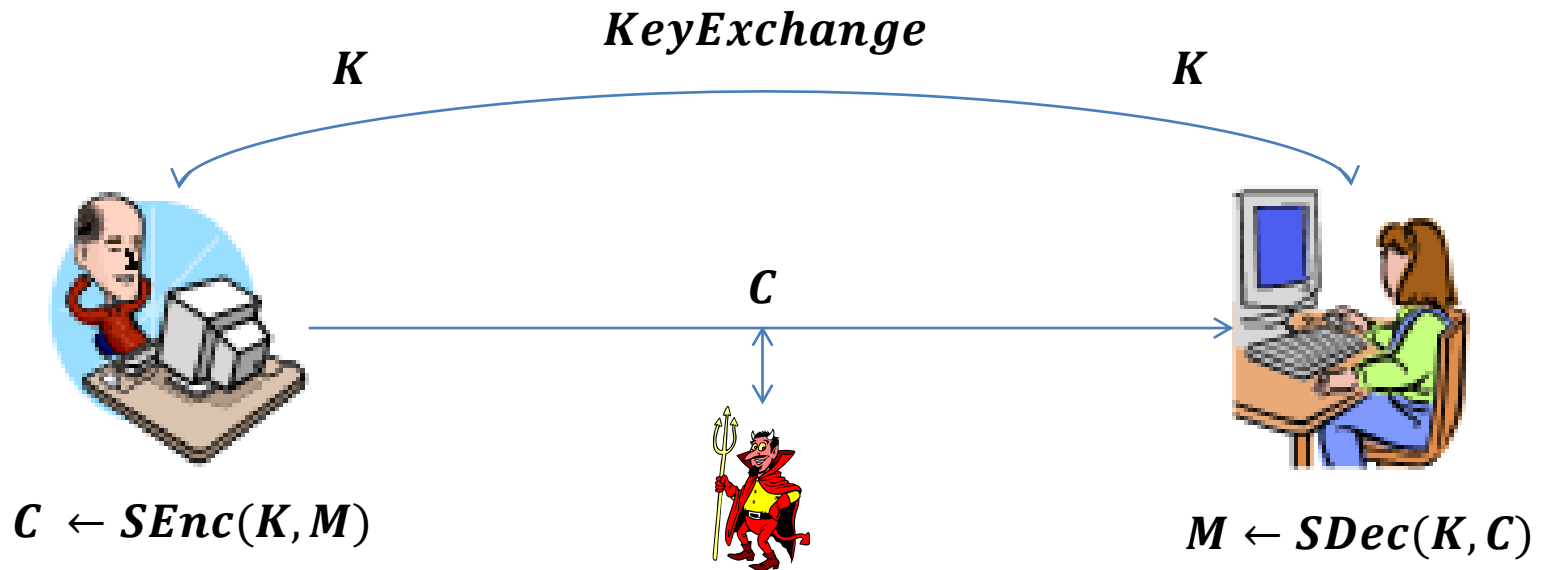
Encryption



Confidentiality: Protect m from A, such that A can't read m

- Symmetric Key Encryption
- Asymmetric Key Encryption
- Hybrid Encryption

Symmetric Key Encryption



Indistinguishability: A does not get information on M from C

- Efficient
- Problem: shared keys

Computational security

- In contrast to the “perfect” security, most cryptosystems have “computational” security
 - This means that it’s certain they can be broken, given enough work by Eve
 - How much is “enough”?
 - At **worst**, Eve tries every key
 - How long that takes depends on how long the keys are
 - But it only takes this long if there are no “shortcuts”!

Some context on what's “enough”

- One modern computer can try about 17 million keys per second
- A medium-sized company or research lab may have 100 computers
- The BOINC project (the largest computing grid in the world) has the computation power of about 300,000 computers

Key size = 40 bits

This was the US legal export limit for a long time

$2^{40} = 1,099,511,627,776$ possible keys

- One computer: 18 hours
- One lab: 11 minutes
- BOINC: 200 ms

Key size = 56 bits

- This was the US government standard (DES) for a long time
- $2^{56} = 72,057,594,037,927,936$ possible keys
- One computer: 134 years
- One lab: 16 months
- BOINC: 4 hours

Key size = 128 bits

- This is the modern standard
- $2^{128} =$
340,282,366,920,938,463,463,374,607,431,768,211,456
- One computer: 635 thousand million million million years
- One lab: 6 thousand million million million years
- BOINC: 2 million million million years
- The Universe is ≈ 13 thousand million years old

Key Exchange

- How do Alice and Bob share the secret key?
 - Meet in person
 - Diplomatic courier
 - ...
 - In general this is very hard
- Or, we invent new technology...

Key Exchange

- Two ways possible:
 - Diffie-Hellman (DH) key exchange
 - Public (or asymmetric) key encryption

DH exchange

- A way to derive the same key between two parties by sharing information.
 - Third party will not be able to derive the key even when it can observe this shared information.
- Assumptions:
 - Alice and Bob have some common component that they share over the insecure channel.
 - There is a secret component that is not shared over the channel.

DH exchange

- Alice

Secret: a

exchanged: $g^a = A$

Calculation: B^a

$$= (g^a)^b$$

Shared key = (g^{ab})

- Bob

Secret: b

exchanged: $g^b = B$

Calculation: A^b

$$= (g^b)^a$$

Shared key = (g^{ab})

DH exchange

Alice and Bob agree on: modulus (17) and base (3)

- Alice

Secret: 5

exchanged: $3^5 \bmod 17$
 $= 243 \bmod 17$
 $= 5$

Calculation: $11^5 \bmod 17$
 $= 161051 \bmod 17$
Key = 10

- Bob

Secret: 7

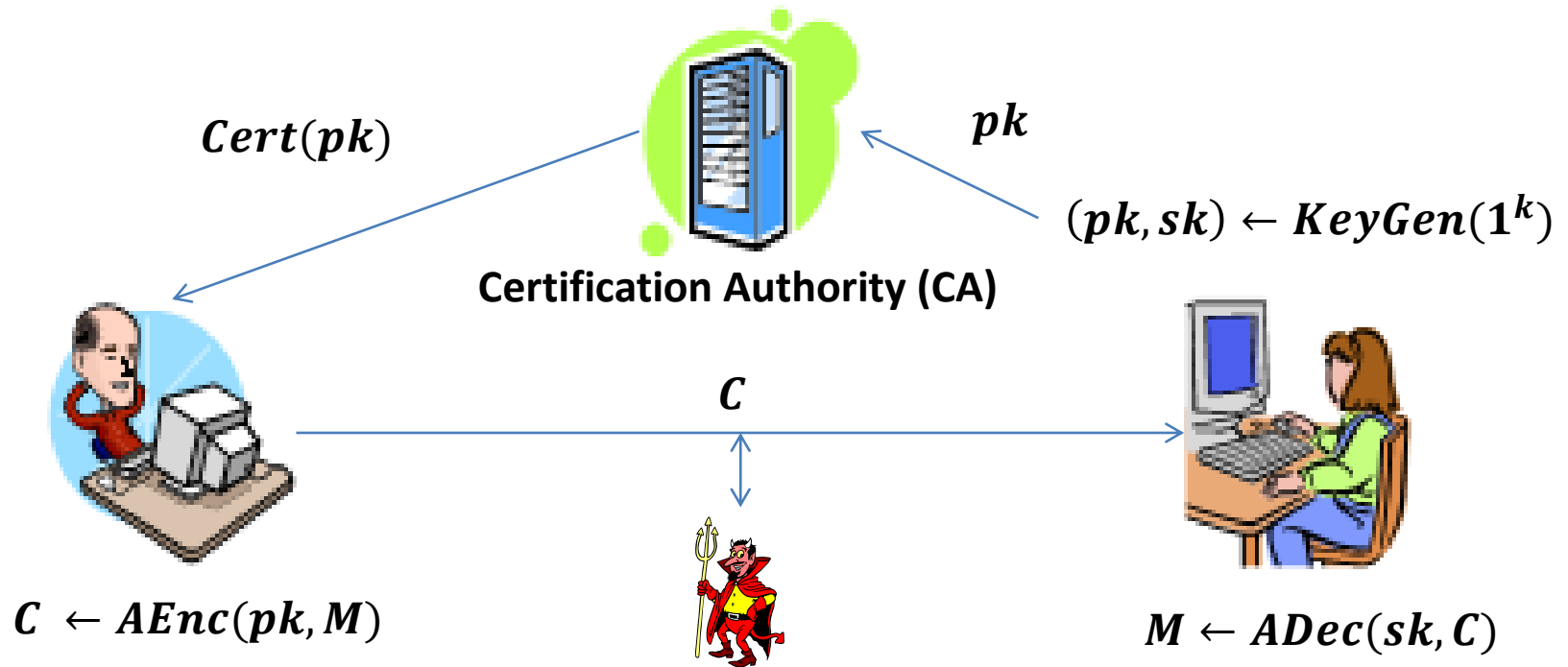
exchanged: $3^7 \bmod 17$
 $= 2187 \bmod 17$
 $= 11$

Calculation: $5^7 \bmod 17$
 $= 78125 \bmod 17$
Key = 10

DH exchange

- Safeguards from a passive adversary.
- But, what about an active adversary?
 - Man-in-the-middle

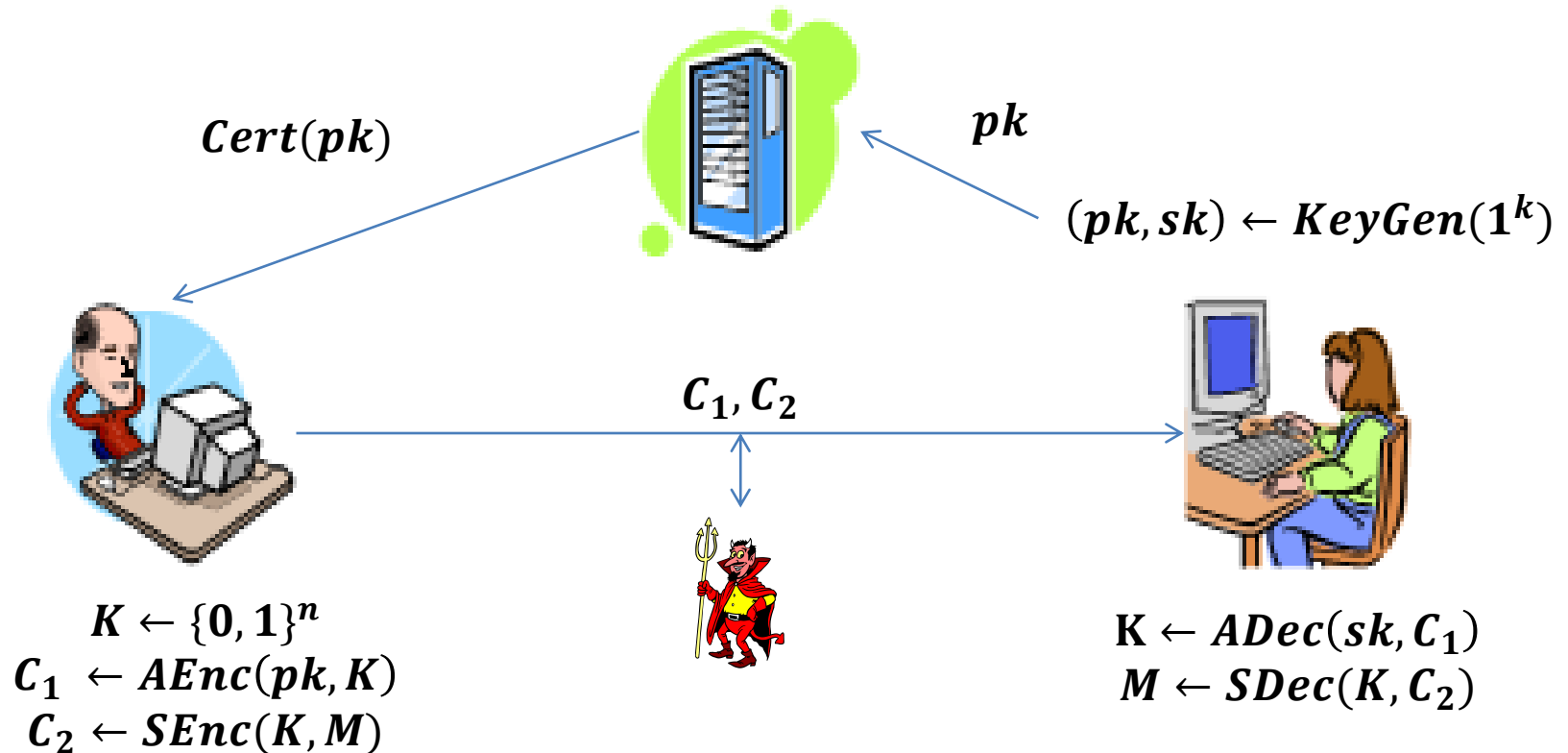
Asymmetric Key Encryption



Indistinguishability: A does not get information on M from C

- Less Efficient

Hybrid Encryption



- Key Distribution of asymmetric key encryption
- Efficiency of symmetric key encryption

Is that it?

- It seems we've got this "sending secret messages" thing down pat.
- What else is there to do?

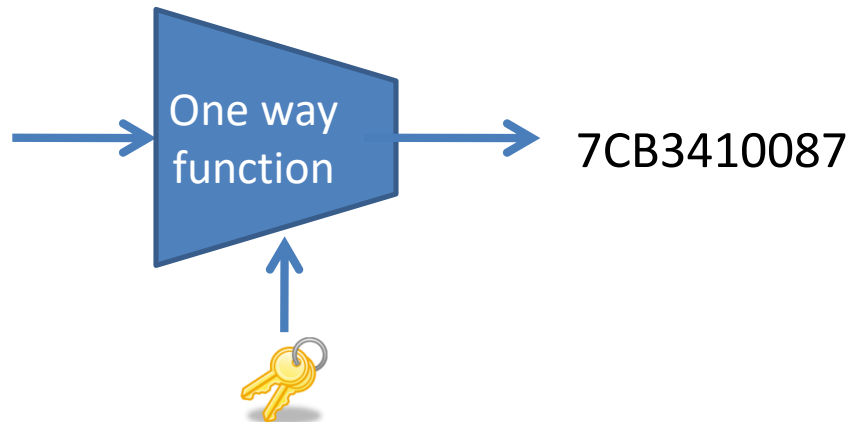
Data Integrity

- How do we tell if a message has changed in transit?
 - Simplest answer: use a checksum
 - For example, add up all the bytes of a message
 - The last digits of serial numbers (credit card, ISBN, etc.) are usually checksums
 - Alice computes the checksum of the message, and sticks it at the end before encrypting it to Bob.
 - When Bob receives the message and checksum, he verifies that the checksum is correct

Data integrity

- Encryption does not protect against modifications
- Message Authentication Code (MAC)
 - Based on a shared secret key
 - Provides data integrity (aka data origin authentication)
 - Same operation done by the sender and the receiver of the message

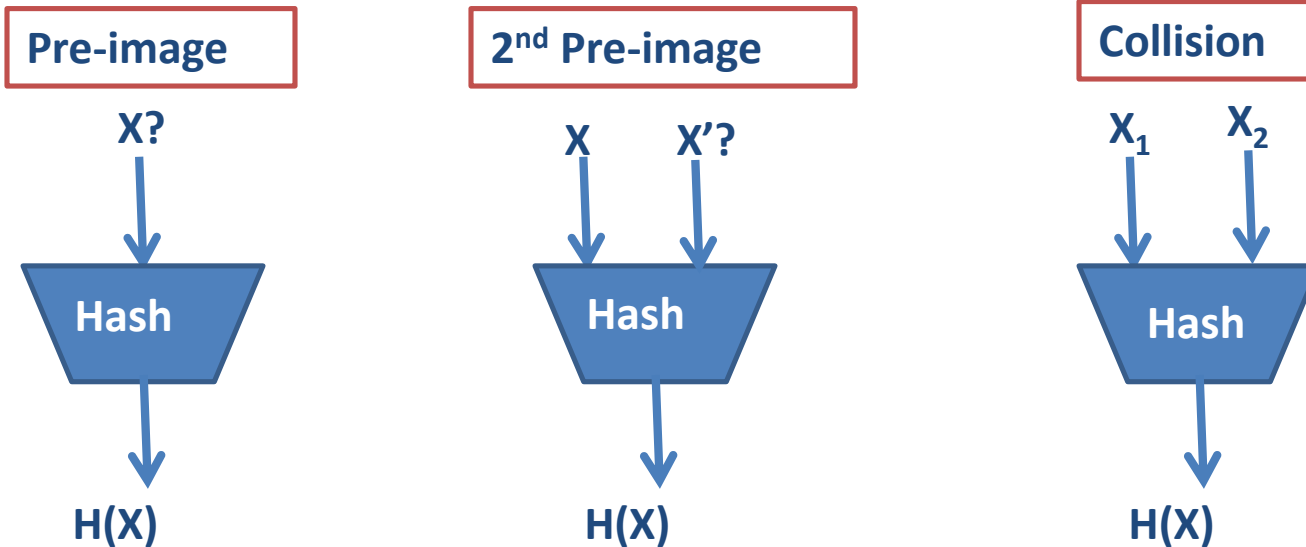
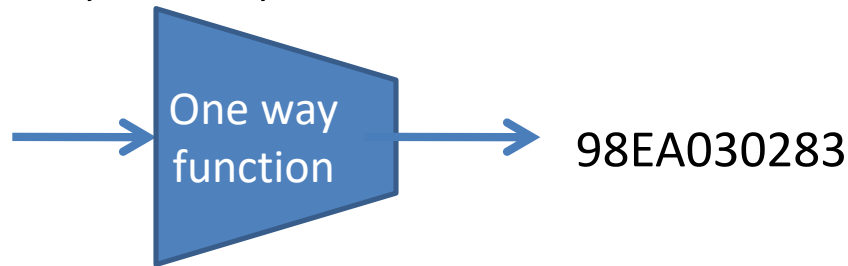
Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Pellentesque vestibulum, nibh et
hendrerit tincidunt, purus quam
pellentesque tortor, in faucibus
tellus arcu iaculis ante.



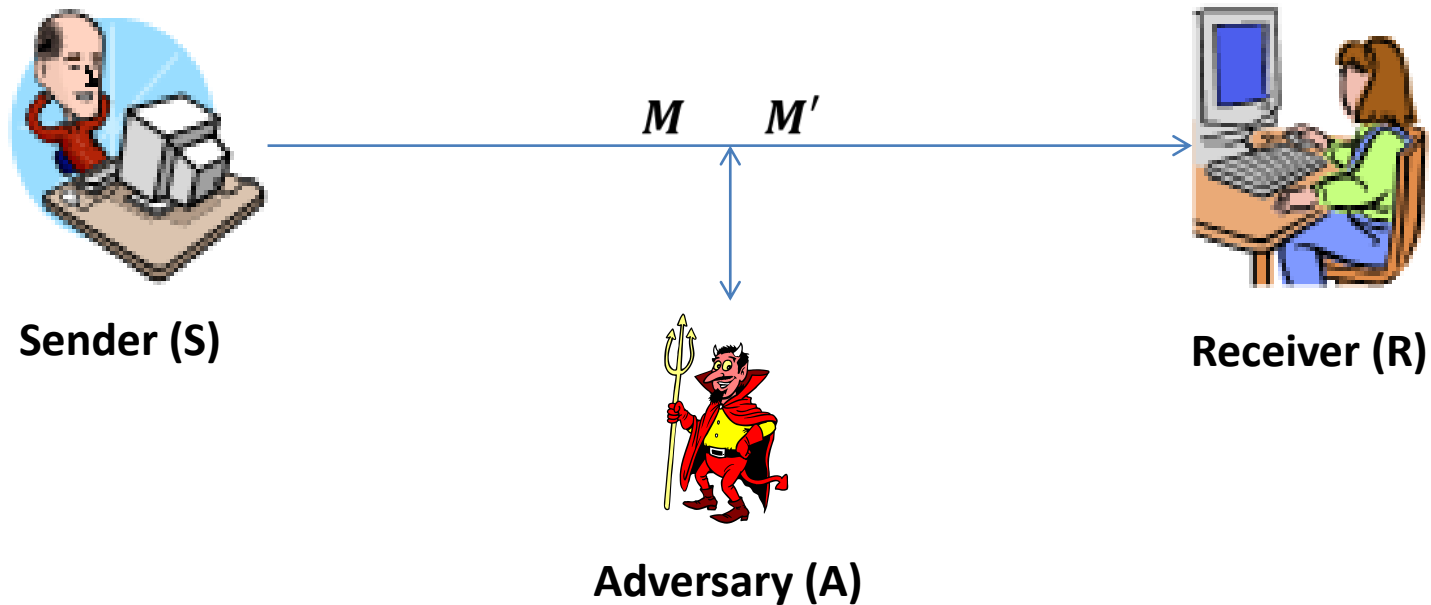
Hash functions

- A hash function h takes an arbitrary length string x and computes a fixed length string $y = h(x)$ called a message digest
 - MD5, RIPEMD, SHA-1, SHA-2, **SHA-3**

Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Pellentesque vestibulum, nibh et
hendrerit tincidunt, purus quam
pellentesque tortor, in faucibus
tellus arcu iaculis ante.

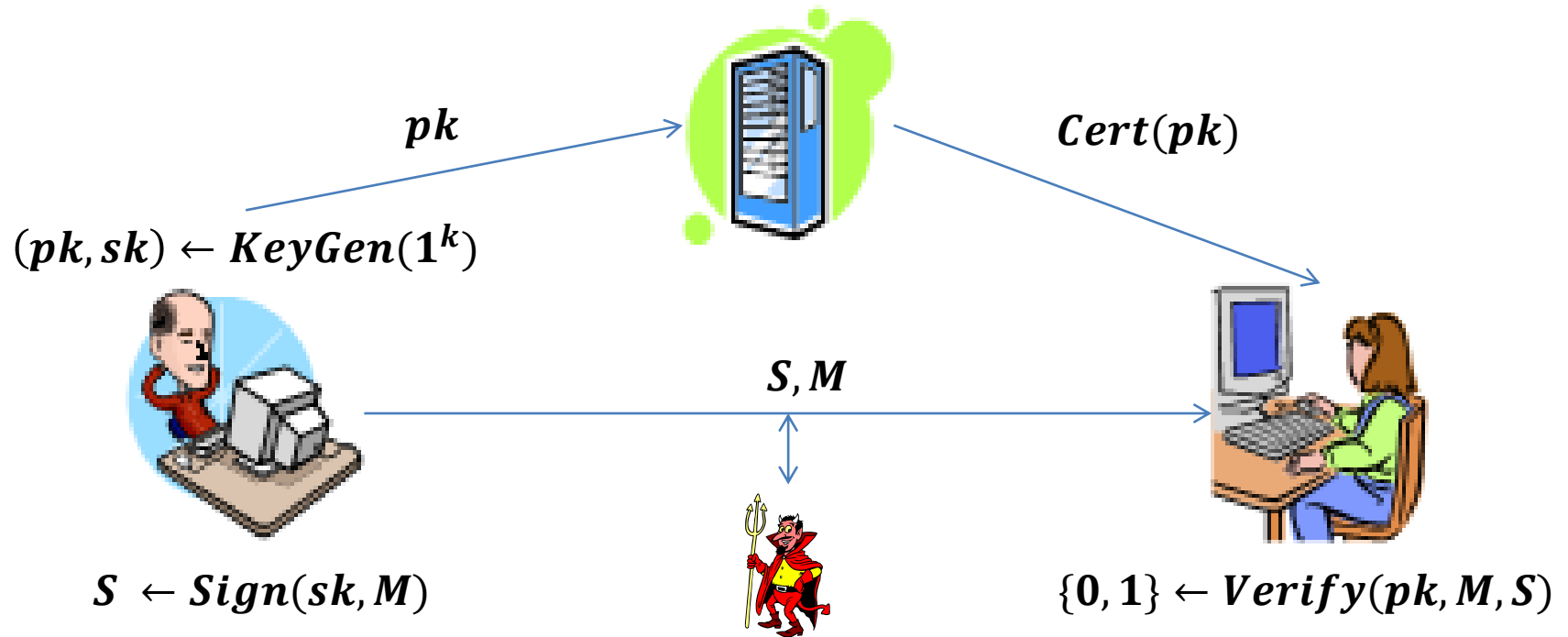


Signatures



- Authenticity: M comes from S
 - Implies integrity: M unmodified

Signatures



Unforgeability: Given pk and $(M_1, S_1, \dots, M_n, S_n)$, A cannot output a signature on a new message $M \neq \{M_1, \dots, M_n\}$

Putting it all together

- We have all these blocks; now what?
- Put them together into protocols
- This is HARD. Just because your pieces all work, doesn't mean what you build out of them will; you have to use the pieces correctly
- Common mistakes include:
 - Using the same stream cipher key for two messages
 - Assuming encryption also provides integrity
 - Falling for replay attacks or reaction attacks
 - LOTS more!

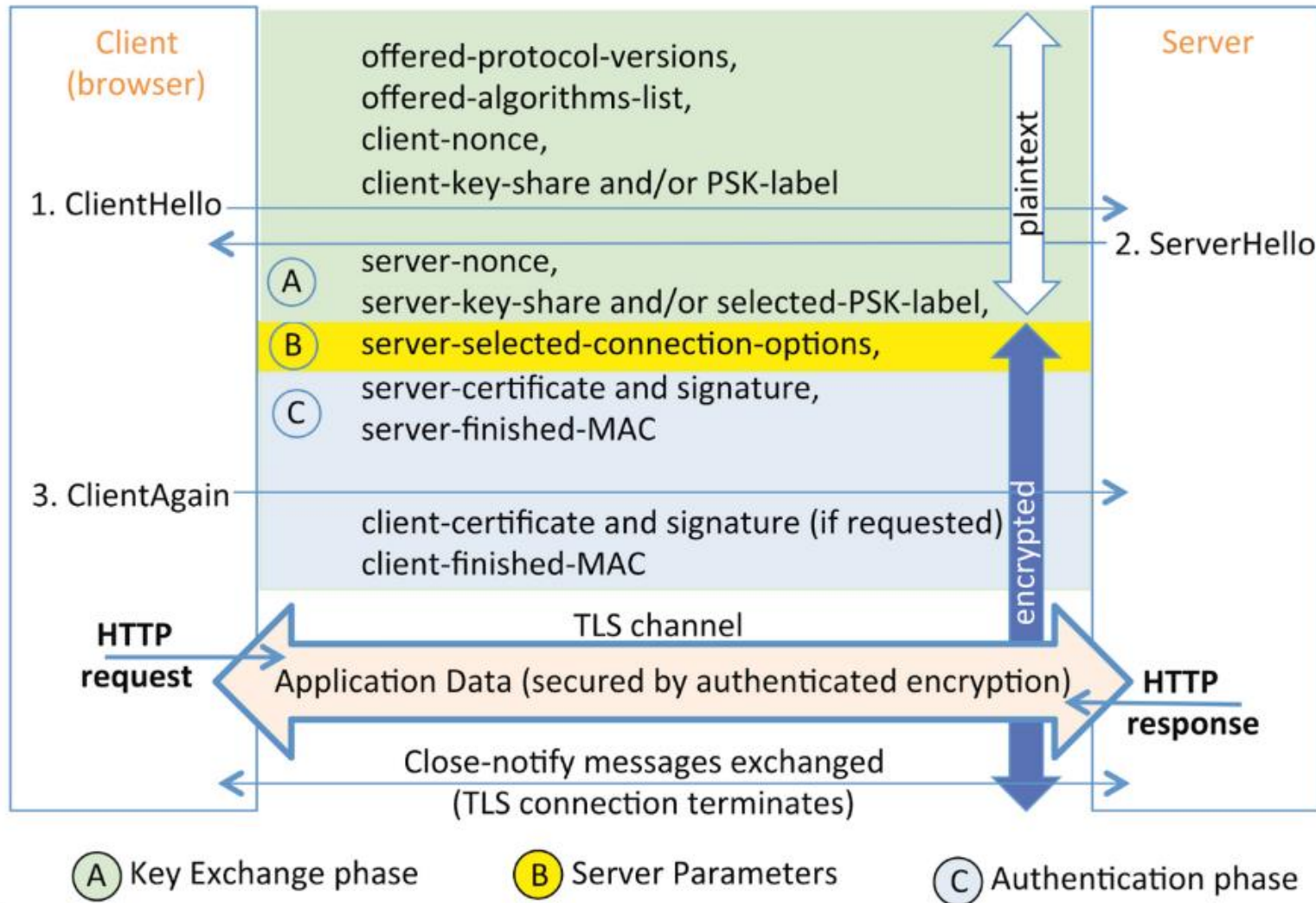
Network security and privacy

- Cryptography is used at every layer of the network stack for both security and privacy applications:
- Link
 - WEP, WPA, WPA2, WPA3
- Network
 - VPN, IPsec
- Transport
 - TLS / SSL, Tor
- Application
 - ssh, Mixminion, PGP, OTR, Signal

TLS/SSL

- In the mid-1990s, Netscape invented a protocol called Secure Sockets Layer (SSL) meant for protecting HTTP (web) connections
 - The protocol, however, was general, and could be used to protect any TCP-based connection
 - HTTP + SSL = HTTPS
- Historical note: there was a competing protocol called S-HTTP. But Netscape and Microsoft both chose HTTPS, so that's the protocol everyone else followed
- SSL went through a few revisions, and was eventually standardized into the protocol known as TLS (Transport Layer Security, imaginatively enough)

TLS connection establishment



Security properties provided by TLS

- Server authentication
- Message integrity
- Message confidentiality
- Client authentication (optional)

Why is client authentication mostly optional?

Application level protocols

- TLS can provide for encryption at the TCP socket level
 - “End-to-end” in the sense of a network connection
 - Is this good enough? Hint: one application may involve multiple TCP connections
- Many applications would like true end-to-end security
 - Human-to-human would be best, but those last 50 cm are really hard!
 - We usually content ourselves with desktop-to-desktop
- We’ll look at some applications:
 - PGP, and instant messaging

Pretty good privacy (PGP)

- The first popular implementation of public-key cryptography.
- Originally made by Phil Zimmermann in 1991
 - He got in a lot of trouble for it, since cryptography was highly controlled at the time.
- Today, there are many (more-or-less) compatible programs
 - GNU Privacy Guard (gpg), Hushmail, etc.

Pretty good privacy

- What does it do?
 - Its primary use is to protect the contents of email messages
- How does it work?
 - Uses public-key cryptography to provide:
 - Encryption of email messages (using hybrid encryption)
 - Digital signatures on email messages (hash-then-sign)

PGP

Pretty Good Privacy (PGP) is a cryptographic system used to secure email and files. It provides:

- **Confidentiality** (encryption)
- **Authentication** (proves who sent the message)
- **Integrity** (detects modifications)
- **Non-repudiation** (the sender cannot later deny sending a signed message)

Recall on public-key cryptography

- In order to use public-key encryption and digital signatures, Alice and Bob must each have:
 - A public encryption key
 - A private decryption key
 - A private signature key
 - A public verification key

Sending a message

- To send a message to Bob, Alice will:
 - Write a message
 - Sign it with her own signature key
 - Encrypt both the message and the signature with Bob's public encryption key
- Bob receives this, and:
 - Decrypts it using his private decryption key to yield the message and the signature
 - Uses Alice's verification key to check the signature

Back to PGP

- PGP's main functions:
 - Create these four kinds of keys
 - encryption, decryption, signature, verification
 - Encrypt messages using someone else's encryption key
 - Decrypt messages using your own decryption key
 - Sign messages using your own signature key
 - Verify signatures using someone else's verification key
 - Sign other people's keys using your own signature key

Obtaining keys

- Earlier, we said that Alice needs to get an authentic copy of Bob's public key in order to send him an encrypted message.
- How does Alice do this?
 - Certificate authorities (CAs)?
- What if we don't involve CAs?
 - Bob could put a copy of his public key on his webpage
 - Is this good enough?

Verifying public keys

- If Alice knows Bob personally, she could:
 - Download the key from Bob's web page
 - Phone up Bob, and verify she's got the right key
 - Problem: keys are big and unwieldy!

```
mQGiBDi5qEURBADitpDzvzW+9lj/zYgK78G3D76hvvtIT6gpTIlwg6WIJNLKJat
01yNpMIYNvpwi7EUd/1SNl6t1/A022p7s7bDbE4T5NJda0IOAgWeOZ/plIJC4+o2
tD2RNuSkwDQcxzm8KUNZOJla4LvGRkm/oUubxyeY5omus7hcfNrBOwjC1wCg4Jnt
m7s3eNfMu72Cv+6FzBgFog8EANirkNdC1Q8oSMDihWj1ogiWbBz4s6HMxzAaqNf/
rCJ9qoK5SLFeoB/r5ksRWty9QKV4VdhHCiy1U2B9tSTlEPYXJHQPZ3mwCxUnJpGD
8UgFM5uKXaEq2pwpArTm367k0tTpMQgXAN2HwiZv//ahQXH4ov30kBBVL5VFxMUL
UJ+yA/4r5HLTpP2SbbqtPWdeW7uDwhe2dTqffAGuf0kuCpHwCTAhr83ivXzT/7OM
```

Fingerprints

- Luckily, there's a better way!
- A fingerprint is a cryptographic hash of a key
- This, of course, is much shorter:
 - B117 2656 DFF9 83C3 042B C699 EB5A 896A 2898
8BF5
- Remember: there's no (known) way to make two different keys that have the same fingerprint, provided that we use a collision-resistant hash function

Fingerprints

- So now we can try this:
 - Alice downloads Bob's key from his webpage
 - Alice's software calculates the fingerprint
 - Alice phones up Bob, and asks him to read his key's actual fingerprint to her
 - If they match, Alice knows she's got an authentic copy of Bob's key
- That's great for Alice, but what about Carol?
 - Carol might not know Bob
 - At least not well enough to phone him

Signing Keys

- Once Alice has verified Bob's key, she uses her signature key to sign Bob's key
- This is effectively the same as Alice signing a message that says,
"I have verified that the key with fingerprint
B117 2656 DFF9 83C3 042B C699 EB5A 896A 2898
8BF5
really belongs to Bob"
- Bob can attach Alice's signature to the key on his webpage
 - If Bob wants, he can get many people to sign his key...
- Can you see some potential issue with key signing?

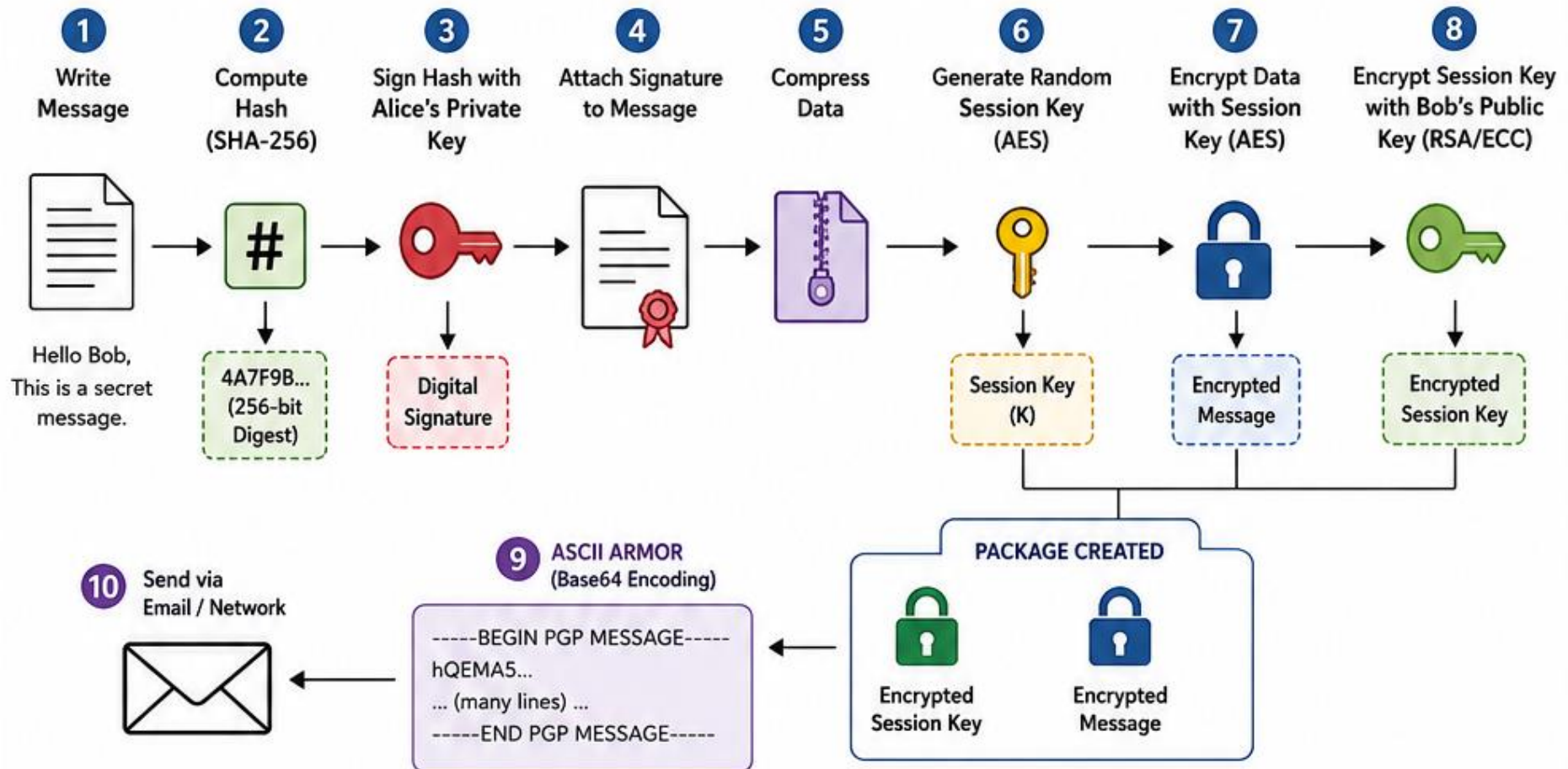
Web of Trust

- Now Alice can act as an introducer for Bob
- If Carol doesn't know Bob, but does know Alice (and has already verified Alice's key, and trusts her to introduce other people):
 - she downloads Bob's key from his website
 - she sees Alice's signature on it
 - she is able to use Bob's key without having to check with Bob personally
- This is called the Web of Trust, and the PGP software handles it mostly automatically

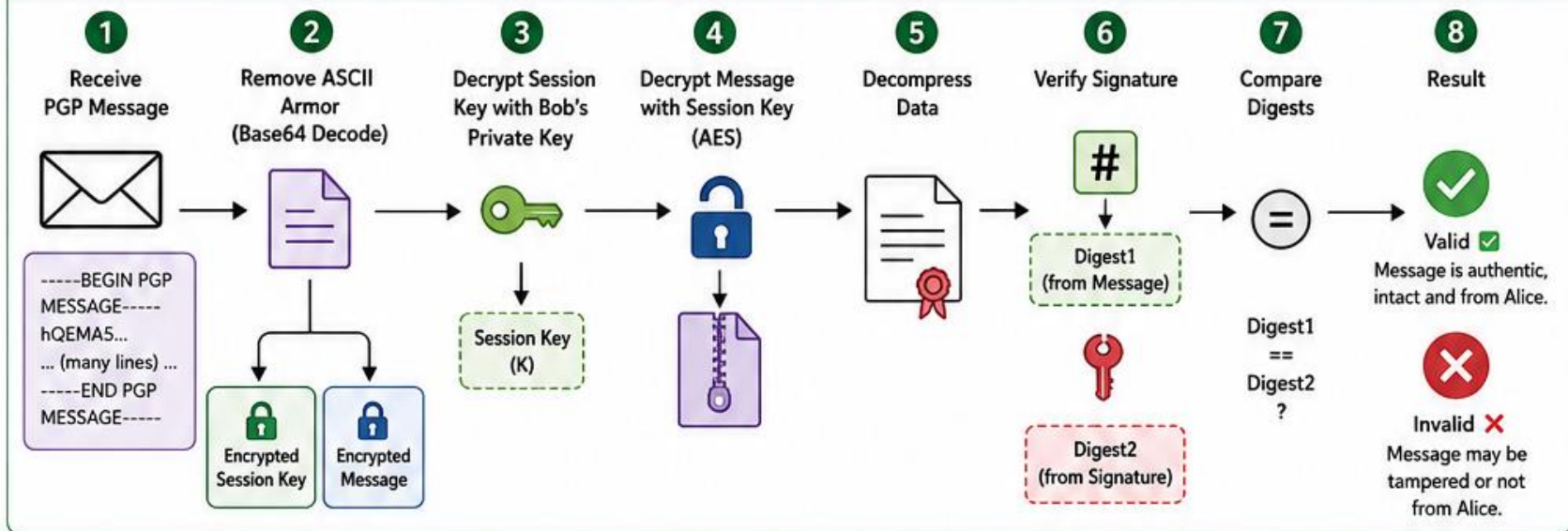
PGP (PRETTY GOOD PRIVACY) – COMPLETE FLOW

Confidentiality • Authentication • Integrity • Non-repudiation

SENDER SIDE – ALICE



RECEIVER SIDE – BOB



WHAT PGP PROVIDES



Confidentiality

Only the intended recipient can read the message.



Authentication

Digital signature proves who sent the message.



Integrity

Any modification to the message will be detected.



Non-repudiation

Sender cannot deny sending the message.

CRYPTOGRAPHIC COMPONENTS USED



Symmetric Encryption (AES)

Encrypts the actual message.



Public-Key Encryption (RSA/ECC)

Encrypts the session key.



Hash Function (SHA-256)

Creates a fixed-size message digest.



Digital Signature

Ensures authentication, integrity and non-repudiation.



Compression

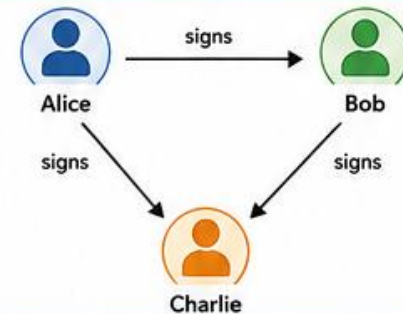
Reduces data size before encryption.



ASCII Armor (Base64)

Converts binary data into text for email transmission.

KEY MANAGEMENT (WEB OF TRUST)



Trust is built by users signing each other's public keys instead of relying on a central Certificate Authority (CA).



WHY SIGN BEFORE ENCRYPT?

- The signature is hidden from attackers because it is encrypted.
- Anyone intercepting the message cannot see or modify the signature.
- After decryption, the recipient can verify the sender.

SIGN → COMPRESS → ENCRYPT → SEND

PGP is a hybrid cryptographic system that combines multiple techniques to deliver strong, efficient, and practical security for email and file communication.

So, great!

- So if Alice and Bob want to have a private conversation by email:
 - They each create their sets of keys
 - They exchange public encryption keys and verification keys
 - They send signed and encrypted messages back and forth
- Pretty Good, no?

Problem 1: Usability

In Proceedings of the 8th USENIX Security Symposium, August 1999, pp. 169-183

Why Johnny Can't Encrypt: A Usability Evaluation of PGP 5.0

Alma Whitten
School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213
alma@cs.cmu.edu

J. D. Tygar¹
EECS and SIMS
University of California
Berkeley, CA 94720
tygar@cs.berkeley.edu

Why Johnny Still Can't Encrypt: Evaluating the Usability of Email Encryption Software

Steve Sheng
Engineering and Public Policy
Carnegie Mellon University
shengx@cmu.edu

Levi Broderick
Electrical and Computer Engineering
Carnegie Mellon University
ljb@ece.cmu.edu

Colleen Allison Koranda
HCI Institute
Carnegie Mellon University
ckoranda@andrew.cmu.edu

Jeremy J. Hyland
Heinz School of Public Policy and
Management
Carnegie Mellon University
jhyland@andrew.cmu.edu

ABSTRACT

Our research seeks to understand the current usability situation of email encryption software, particularly PGP 9 in comparison to previous studies of PGP 5. We designed a pilot study to find current problems in the following areas: create a key pair, get public keys, verify public keys, encrypt an email, sign an email,

email message to test user's response to PGP's automatic decryption.

2. MAJOR FINDINGS

2.1 Verify Keys

We found that key verification and signing is still severely

Why Johnny Still, Still Can't Encrypt: Evaluating the Usability of a Modern PGP Client

Scott Ruoti, Jeff Andersen, Daniel Zappala, Kent Seamons
Brigham Young University
(ruoti, andersen) @ isrl.byu.edu, {zappala, seamons} @ cs.byu.edu

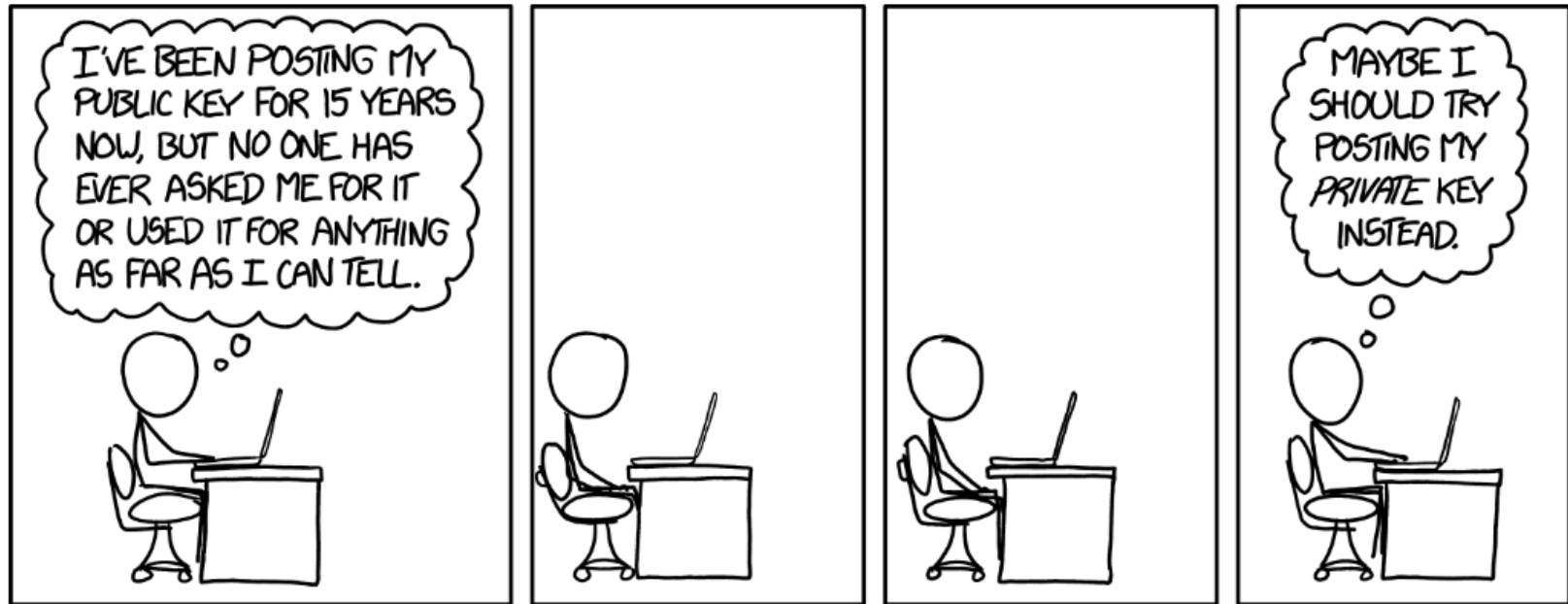
ABSTRACT

This paper presents the results of a laboratory study involving Mailvelope, a modern PGP client that integrates tightly with existing webmail providers. In our study, we brought

In our study of 20 participants, grouped into 10 pairs of participants who attempted to exchange encrypted email, only one pair was able to successfully complete the assigned tasks using Mailvelope. All other participants were unable to com-

- Common mistakes:
 - Encrypt a message with the sender's public key
 - Send private key so that recipient can decrypt a message
- Oftentimes, study participants cannot send a PGP-encrypted e-mail after 45min

Problem 1: Usability



(Public Key)

Problem 2: Key compromise

- Suppose (encrypted) communications between Alice and Bob are recorded by the “bad guys”
 - criminals, competitors, etc
- Later, Bob’s computer is stolen by the same bad guys
 - Or just broken into
 - Virus, trojan, etc
- All of Bob’s key material is recovered

The bad guys can...

- Decrypt past messages
- Learn their content
- Learn that Alice sent them
- And have a mathematical proof they can show to anyone else!

How private is that?

What went wrong?

- Bob's computer got stolen?
- How many of you have never...
 - Left your laptop unattended?
 - Not installed the latest patches?
 - Run software with a remotely exploitable bug?
- What about your friends?

What really went wrong?

- PGP creates lots of incriminating records:
 - Key material that decrypts data sent over the public Internet
 - Signatures with proofs of who said what
- Alice had better watch what she says!
 - Her privacy depends on Bob's actions

Casual conversations

- Alice and Bob talk in a room
- No one else can hear
 - Unless being recorded
- No one else knows what they say
 - Unless Alice or Bob tells them
- No one can prove what was said
 - Not even Alice or Bob
- These conversations are “off-the-record” (OTR)

We like off-the-record conversations

- Legal support for having them
 - Illegal to record other people's conversations without notification
- We can have them over the phone
 - Illegal to tap phone lines
- But what about over the Internet?

Cryptographic tools

- We have the cryptographic tools to do OTR, but we need to have new perspectives on how to use these tools:
 - We want perfect forward secrecy
 - We want future secrecy
 - We want deniable authentication

Perfect forward secrecy

- Future key compromises should not reveal past communication
- Use secret-key encryption with a short-lived key (a session key)
- The session key is created by a modified Diffie-Hellman protocol
- Discard the session key after use
 - Securely erase it from memory (and everywhere possible)
- Use long-term keys only to authenticate the Diffie-Hellman protocol messages only

Future secrecy

- Past key compromises should not compromise the security of future sessions
 - What happens if new session keys are just hashes of the previous key?
 - e.g., derived through a session key ratchet:
 - $K2 = H(K1)$, $K3 = (H(H(K1))) = H(K2)$
- So what can we do?
 - Regularly replace potentially compromised session keys with new key material

Deniable authentication

- Do not want digital signatures
 - Non-repudiation is great for signing contracts, but undesirable for private conversations
- But we do want authentication
 - We can't maintain privacy if attackers can impersonate our friends
- Use Message Authentication Codes(MAC)
 - We talked about these earlier

